

Hi, JTAG!

Joint Test Action Group (JTAG) is the common name for the IEEE 1149.1 Standard Test Access Port and Boundary-Scan Architecture. JTAG was originally introduced around 1990 for testing printed circuit boards (PCBs) using boundary scan, and is still widely used for this application.

Later, JTAG was used as an integrated circuit (IC) debug port in embedded processors to enable debuggers to connect and communicate with chips to perform operations like single stepping, placing hardware and software breakpoints and source-level debugging of code. JTAG can also be used for flashing of boot firmware.

The connection between the host PC and the JTAG debugger is generally established via USB, serial port or Ethernet depending on the model. If it is an Ethernet supported debugger, we can debug the target remotely from anywhere in the network.

JTAG debugging is also referred to as on-chip debugging (OCD). Its working speed is from 10MHz to 100MHz depending on the chips. Some JTAG debuggers have Eclipse plug-ins to access JTAG via an integrated development environment (IDE), while some JTAG interfaces use any GNU debugger (GDB)-aware frontend as software interface.

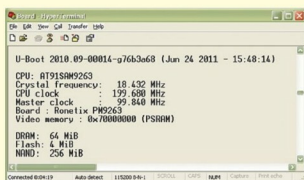


Fig. 2: U-Boot display

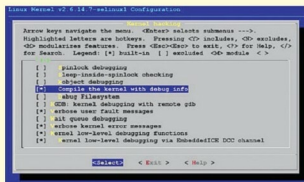


Fig. 3: Linux kernel configuration

kernel is not stabilised, or when these communication devices themselves need to be debugged.

In some scenarios, the Ethernet or UART port may not be available for agent-based debugging. For example, cell phones simply do not have serial Ethernet interfaces!

Developers debugging in user mode will require stepping into system calls from user mode to Linux kernel mode and back into user mode. Keeping track of memory addresses when stepping into and out of these two modes is challenging because of the

usage of virtual memory management in Linux. Keeping track of memory mapping and memory allocation during this process is critical. When using the traditional agent based solution, we need to use both GDB and KGDB to trace system calls into the Linux kernel and kernel modules. The use of multiple agent based debug tools may complicate the debug process.

These are just some of the problems one could face when trying to solve problems in embedded Linux based systems using traditional non-JTAG methods. Now, let us

look at how things become simpler with Linux-aware JTAG debuggers.

Busting boot-loader issues

A boot-loader loads and starts the Linux kernel every time the system is powered on or reset. Boot-loaders, such as the open source U-Boot, support a wide range of embedded processor architectures and boards.

In order to set up a boot-loader, we need to configure hundreds of registers. Even if a single register initialisation value is wrong, it may cause issues

while booting up the Linux kernel or later while running Linux processes. The boot-loader passes on arguments while calling the Linux kernel for execution and any mistakes in the arguments may prevent Linux from booting up.

Agent based debuggers cannot be used to debug these issues because the Linux OS is not up during this stage. To debug these issues we need to have access to the processor registers, for reading the present values, and overwriting them with correct values in case they are wrong. This can be done only using JTAG debugging tools.

Readying your debuggers

Most JTAG debuggers need to be provided a configuration file that has information like processor architecture, JTAG clock, core start-up mode, memory initialisation, etc. There is no standard format for these configuration files, so every JTAG debugger uses its own format. However, developers do not have to worry much about these configuration files because the vendors who provide the JTAG debuggers will also share these files for all supported processors and most of the commonly-available evaluation boards in the market. For custom boards, the users can edit an existing configuration file that more or less matches the board features and use.

The configuration file is very important because it helps the debugger to know the processor we are trying to debug, the components present on this target board, and how all these components are interconnected. If these values are correctly set, the JTAG debugger can load the Linux OS even without a boot-loader. This helps when the boot-loader and the OS are being developed simultaneously.

Source-level debugging using JTAG

For source-level debugging, the Linux kernel needs to be built with debug info. Select the option 'Kernel Hacking -> Compile the kernel with debug info' in the configuration file for building